LAB SESSION 11

Doubly Linked List – Deletion Operations

THEORY

A Doubly Linked List (DLL) allows deletion from both ends and from any specific position with relative ease compared to singly linked lists, thanks to its two-pointer structure (prev and next). In deletion operations, the key task is updating the links of neighboring nodes to remove the target node from the chain without breaking the sequence.
Each deletion case requires careful handling of pointers:

1. **Deletion at Beginning** – Update head to the second node, set its prev to NULL, and free the removed node's memory.
2. **Deletion at End** – Move to the last node, update the second-last node's next to NULL, and free the last node.
3. **Deletion at a Specific Position** – Adjust the next of the previous node and the prev of the next node to bypass the deleted node.

Advantages of DLL Deletion:

- Can remove elements from either end in constant time if head and tail pointers are maintained.
- Easier mid-list deletions since we have direct access to the previous node.

Common Pitfalls:

- Forgetting to handle empty list scenarios.
- Not updating both prev and next links.
- Memory leaks if the deleted node's memory is not freed.

PROCEDURE

1. Define a Node structure with data, prev, and next.
2. Implement:
3. deleteAtBeginning()
4. deleteAtEnd()
5. deleteAtPosition()
6. Handle cases for an empty list or invalid positions.
7. Test all deletion cases with sample data.

CODE

```
#include <iostream>
using namespace std;

struct Node {
   int data;
   Node* prev;
   Node* next;
};

Node* head = NULL;
```

```
void insertAtEnd(int value) {
    Node* newNode = new Node();
    newNode->data = value;
    newNode->next = NULL;
    if (head == NULL) {
        newNode->prev = NULL;
        head = newNode;
        return;
    }
    Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
    newNode->prev = temp;
}

void deleteAtBeginning() {
    if (head == NULL) {
        cout << "List is empty.\n";
        return;
    }
    Node* temp = head;
    head = head->next;
    if (head != NULL)
        head->prev = NULL;
    delete temp;
}

void deleteAtEnd() {
    if (head == NULL) {
        cout << "List is empty.\n";
        return;
    }
    Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    if (temp->prev != NULL)
        temp->prev->next = NULL;
    else
        head = NULL; // Only one node
    delete temp;
}

void deleteAtPosition(int pos) {
    if (head == NULL) {
        cout << "List is empty.\n";
        return;
    }
    Node* temp = head;
```

```
    for (int i = 0; i < pos && temp != NULL; i++) {
        temp = temp->next;
    }
    if (temp == NULL) {
        cout << "Position out of range.\n";
        return;
    }
    if (temp->prev != NULL)
        temp->prev->next = temp->next;
    else
        head = temp->next; // Deleting first node
    if (temp->next != NULL)
        temp->next->prev = temp->prev;
    delete temp;
}

void displayForward() {
    Node* temp = head;
    while (temp != NULL) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
}

int main() {
    insertAtEnd(10);
    insertAtEnd(20);
    insertAtEnd(30);
    insertAtEnd(40);
    cout << "Original List: ";
    displayForward();

    deleteAtBeginning();
    cout << "After Deleting Beginning: ";
    displayForward();

    deleteAtEnd();
    cout << "After Deleting End: ";
    displayForward();

    deleteAtPosition(0);
    cout << "After Deleting Position 0: ";
    displayForward();

    return 0;
}
```

EXPECTED OUTPUT

Original List: 10 20 30 40
After Deleting Beginning: 20 30 40
After Deleting End: 20 30
After Deleting Position 0: 30

EXERCISE

1. Modify deleteAtPosition() to handle deletion from the last position without calling deleteAtEnd().
2. Write a function to delete all nodes with a given value.
3. Implement a function to clear the entire list.
4. Write a function to count and return the number of nodes after each deletion.